

LESSON 7

BFS, DFS & Graphs

Graph traversal = Path planning in robotics. A*, Dijkstra extend BFS.

- 7.1 Graph Representation
- 7.2 BFS (Breadth First Search)
- 7.3 DFS (Depth First Search)
- 7.4 Shortest Path with BFS
- 7.5 Grid BFS (Robot Navigation)
- 7.6 Number of Islands (Classic)
- 7.7 A* and Dijkstra Overview

7.1 Graph Representation

```
# Adjacency List (most common in interviews)
graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"]
}
# graph["A"] gives all neighbors of A

# Build from edge list
edges = [(0,1), (0,2), (1,3), (2,3)]
graph = {}
for u, v in edges:
    graph.setdefault(u, []).append(v)
    graph.setdefault(v, []).append(u) # undirected
```

7.2 BFS - Breadth First Search

Uses **queue**. Explores level by level. Guarantees **shortest path** in unweighted graphs.

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    result = []

    while queue:
        node = queue.popleft() # take from FRONT
        result.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor) # add to BACK

    return result
```

```

bfs(graph, "A")

Queue: [A]           Visit A. Add B,C
Queue: [B, C]        Visit B. Add D,E
Queue: [C, D, E]     Visit C. Add F
Queue: [D, E, F]     Visit D. Nothing new
Queue: [E, F]        Visit E. F already visited
Queue: [F]           Visit F. Nothing new

Result: [A, B, C, D, E, F] (level by level!)
Level 0: A
Level 1: B, C
Level 2: D, E, F

```

7.3 DFS - Depth First Search

```

# Iterative (using stack)
def dfs(graph, start):
    visited = set()
    stack = [start]
    result = []

    while stack:
        node = stack.pop()           # take from TOP
        if node not in visited:
            visited.add(node)
            result.append(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)
    return result

# Recursive (more natural)
def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)

```

7.4 Shortest Path with BFS

```
def shortest_path(graph, start, end):
    queue = deque([(start, [start])]) # (node, path_so_far)
    visited = set([start])

    while queue:
        node, path = queue.popleft()

        if node == end:
            return path # found shortest!

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None # no path
```

7.5 Grid BFS (Robot Navigation!)

Treat each cell as a node. 4 neighbors = up, down, left, right.

```
from collections import deque

def shortest_path_grid(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    queue = deque([(start[0], start[1], 0)]) # row, col, dist
    visited = set()
    visited.add((start[0], start[1]))

    # 4 directions: right, left, down, up
    directions = [(0,1), (0,-1), (1,0), (-1,0)]

    while queue:
        row, col, dist = queue.popleft()

        if (row, col) == end:
            return dist # found shortest distance!

        for dr, dc in directions:
            nr, nc = row + dr, col + dc
            if (0 <= nr < rows and 0 <= nc < cols
                and grid[nr][nc] != 1 # not a wall
                and (nr, nc) not in visited):
                visited.add((nr, nc))
                queue.append((nr, nc, dist + 1))

    return -1 # no path exists
```

7.6 Number of Islands (LeetCode #200 - Classic)

```

# Grid: 1=land, 0=water. Count distinct islands.
# An island is a group of connected 1s.

def count_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    count = 0

    def dfs(r, c):
        # Out of bounds or water? Stop.
        if (r < 0 or r >= rows or c < 0 or c >= cols
            or grid[r][c] == "0"):
            return
        grid[r][c] = "0" # mark as visited (sink it)
        dfs(r+1, c)      # down
        dfs(r-1, c)      # up
        dfs(r, c+1)      # right
        dfs(r, c-1)      # left

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                count += 1 # new island found!
                dfs(r, c)  # mark entire island

    return count

```

```

grid = [
    ["1","1","0","0","0"],
    ["1","1","0","0","0"],
    ["0","0","1","0","0"],
    ["0","0","0","1","1"]
]

Scan -> (0,0)="1" -> new island! count=1. DFS marks all connected:
    (0,0),(0,1),(1,0),(1,1) all become "0"
Scan -> (2,2)="1" -> new island! count=2. DFS marks (2,2)
Scan -> (3,3)="1" -> new island! count=3. DFS marks (3,3),(3,4)

Answer: 3 islands

```

7.7 BFS vs DFS & Algorithm Overview

	BFS	DFS
Structure	Queue (FIFO)	Stack (LIFO) / Recursion
Order	Level by level	Go deep, then backtrack
Best For	Shortest path	Explore all, detect cycles
Space	O(width of graph)	O(depth of graph)
Robotics	Path planning, wavefront	Maze solving, mapping

Algorithm	What It Does	When to Use
BFS	Shortest path, unweighted	Equal cost edges
Dijkstra	Shortest path, weighted	Different edge costs
A*	Dijkstra + heuristic	When you know goal location
RRT/RRT*	Random sampling planner	High-dimensional, complex